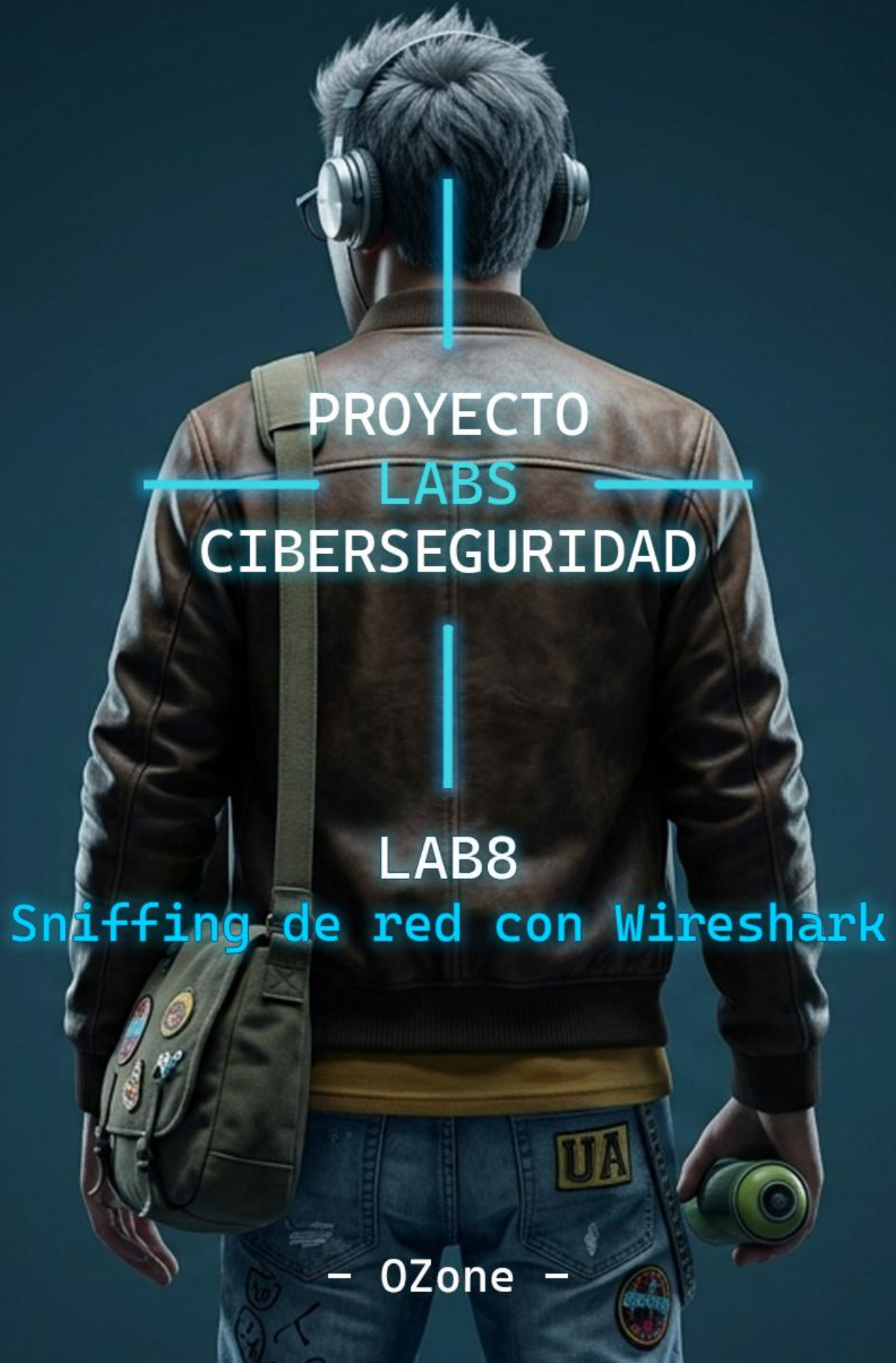


Ruben Apablaza Muñoz



PROYECTO
LABS
CIBERSEGURIDAD

LAB8

Sniffing de red con Wireshark

- OZone -

⚠ **AVISO LEGAL Y DESCARGO DE RESPONSABILIDAD** ⚠

Este documento tiene fines **estrictamente educativos y de formación profesional** en el área de la ciberseguridad. El autor no se hace responsable por el mal uso de la información, scripts o comandos aquí descritos.

- **Entornos de prueba**

Se recomienda ejecutar este laboratorio exclusivamente en entornos controlados (Máquinas Virtuales o Contenedores). Nunca ejecutes scripts o comandos desconocidos en servidores de producción sin antes probarlos y entender su funcionamiento.

- **Responsabilidad del usuario**

El uso de herramientas de nivel administrativo (`sudo`) conlleva riesgos. El lector es el único responsable de cualquier pérdida de datos o interrupción del servicio que pudiera derivarse de la aplicación de este manual.

- **Ética profesional**

Las técnicas de análisis aquí descritas deben ser utilizadas siempre bajo un marco legal y con la autorización debida.

Contenido

Requisito de operación	4
1. Preparar la interfaz en Kali.....	7
2. Abrir Wireshark en la interfaz correcta	11
3. ICMP (tráfico normal).....	13
4. SSH (tráfico cifrado).....	16
5. FTP inseguro desde Metasploitable2.....	21
6. TELNET totalmente inseguro (también por eth1)	24
6.1 DIFERENCIA ENTRE PROTOCOLOS MODERNOS Y ANTIGUOS	28
7. HTTP sin cifrado (opcional si Ubuntu tiene Apache)	29
Conclusión.....	33

Introducción

Bienvenido al Laboratorio 8

Tras haber fortalecido nuestras defensas en los laboratorios previos mediante el *hardening* de sistemas y la configuración de firewalls, es momento de cambiar la perspectiva. Para defender una red de manera efectiva, primero debemos entender qué es lo que un atacante puede "ver" cuando logra posicionarse dentro de ella.

Este laboratorio se centra en el **sniffing de red**, una técnica de interceptación de datos que nos permite inspeccionar el tráfico que circula por nuestras interfaces. Utilizando **Wireshark**, la herramienta de análisis de protocolos más importante de la industria, realizaremos una autopsia en tiempo real de nuestras comunicaciones.

El objetivo principal no es solo capturar paquetes, sino desarrollar un criterio técnico para diferenciar entre el tráfico legítimo y seguro (como SSH) y la exposición crítica que suponen los protocolos heredados (como FTP o Telnet).

A lo largo de esta guía, validaremos por qué el cifrado no es un lujo, sino una necesidad básica, al observar cómo las credenciales y comandos pueden quedar expuestos en texto plano ante cualquier observador malintencionado en la red.

Requisito de operación

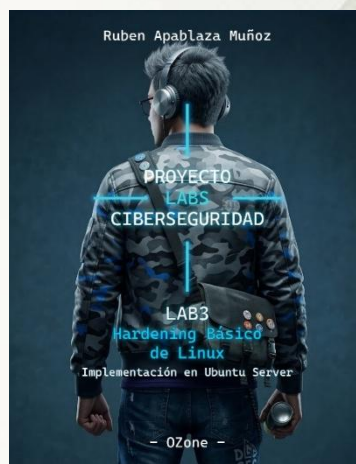
Esta guía es la continuación estratégica de los laboratorios:

- Lab 1: Instalación y preparación de Ubuntu Server.
- Lab 2: Análisis táctico de logs en sistemas Linux.
- Lab 3: Hardening Básico de Linux.
- Lab 4: Hardening AVANZADO de Linux: UFW + Fail2Ban + CrowdSec + RK Hunter.
- Lab 5: Instalación de Kali Linux
- Lab 6: Instalación de Metasploitable2
- Lab 7: Escaneo de red con Nmap

En ciberseguridad, la integridad del entorno es fundamental. Para garantizar que las acciones que vamos a realizar hoy sean consistentes y que las configuraciones de red no interfieran con tus hallazgos, es **altamente recomendable** haber completado la fase previa.

Si aún no tienes tu entorno listo o quieres asegurarte de que tu máquina cumple con los estándares de este proyecto, puedes realizar ambos laboratorios aquí:

Haz *click* en la imagen para acceder al **Lab** correspondiente.



PROYECTO 8: Sniffing de red con Wireshark

Objetivo:

Aprender a:

- Capturar tráfico real de la red
- Identificar protocolos inseguros
- Ver credenciales en texto plano
- Entender cómo se ven los ataques más básicos
- Conocer el tráfico normal vs anormal

Maquinas necesarias:

- **Kali Linux** (tiene Wireshark)
- **Ubuntu Server** (generara tráfico)
- **Metasploitable 2** (tráfico inseguro perfecto para analizar)

1. Preparar la interfaz en Kali

En Kali:

```
ip -4 a
```

Como muestra la imagen a continuación mi Kali no tiene ip asignada en la interfaz **eth0** que en mi caso es la salida a Internet por Bridge

```
(ozone@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether : brd ff:ff:ff:ff:ff:ff  
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether : brd ff:ff:ff:ff:ff:ff  
    inet 192.168.118.128/24 brd 192.168.118.255 scope global dynamic noprefixroute eth1  
        valid_lft 1253sec preferred_lft 1253sec  
    inet6 :/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

Por lo tanto tendré que solicitar una nueva ip por medio del comando

```
sudo dhcpcd (nombre de la interfaz)
```

```
(ozone@kali)-[~]  
$ sudo dhclient eth0  
[sudo] password for ozone:
```

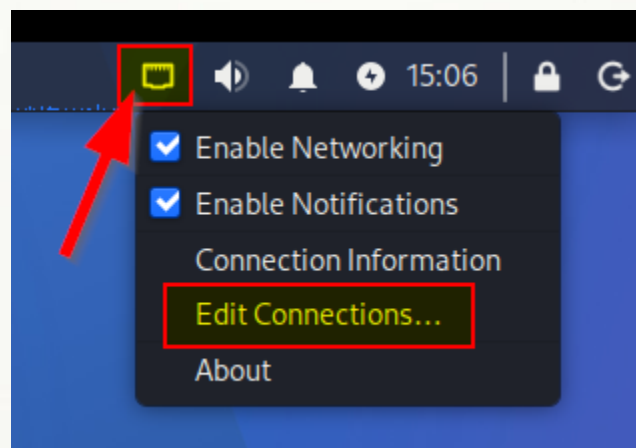


Y ahora si vuelvo a ejecutar **ip a** se puede ver como ya tengo una dirección ip asignada

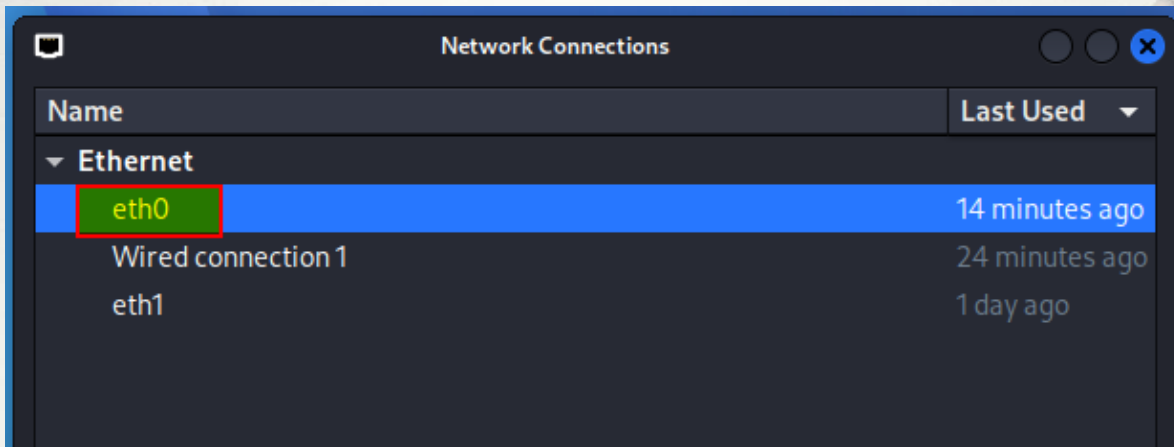
```
(ozone@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:00:00:00 brd ff:ff:ff:ff:ff:ff
   inet 192.168.0.38/24 brd 192.168.0.255 scope global dynamic eth0
       valid_lft 604637sec preferred_lft 604637sec
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:00:00:00 brd ff:ff:ff:ff:ff:ff
   inet 192.168.118.128/24 brd 192.168.118.255 scope global dynamic noprefixroute eth1
       valid_lft 1063sec preferred_lft 1063sec
   inet6 ::1/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Si observamos la imagen anterior podemos notar que solo recibo dirección IPv4 y no IPv6 en la interfaz **eth0**... en caso de que necesitamos que se asigne una dirección IPv6 por DHCP haremos lo siguiente:

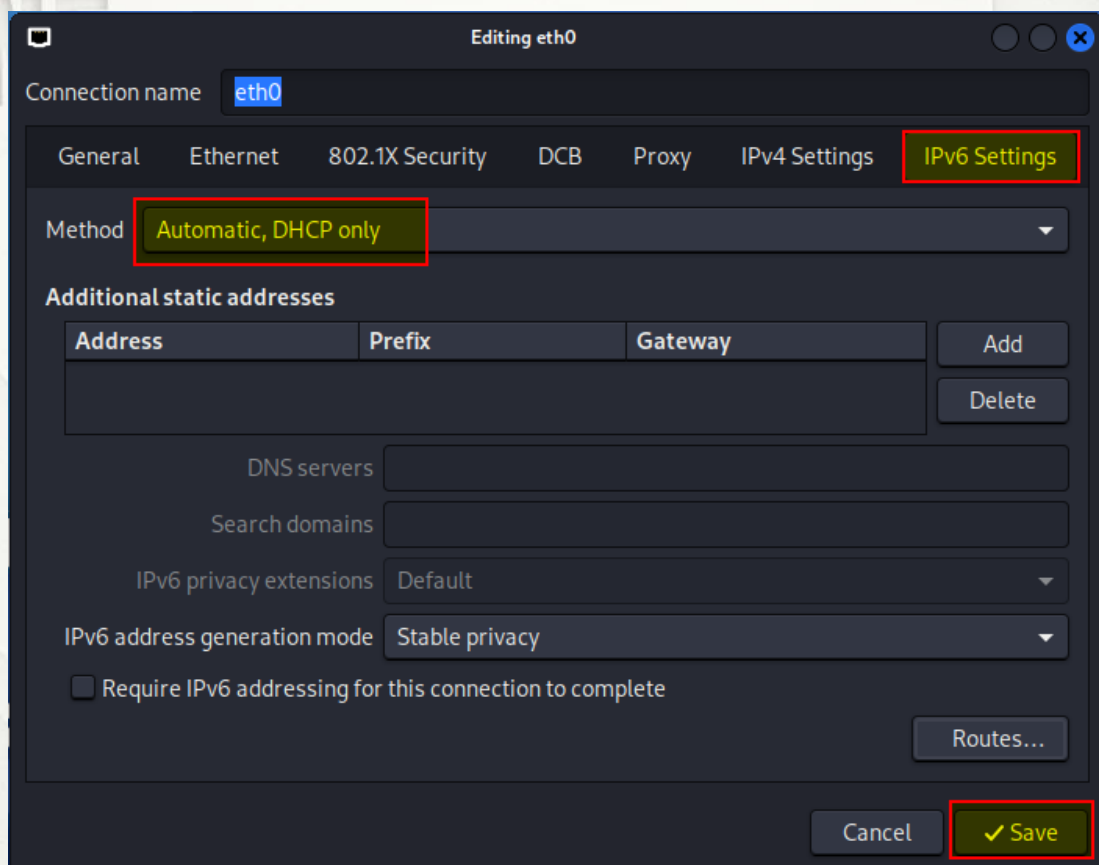
Haremos click derecho en el símbolo de red y después ingresamos a la opción **“Edit Connections...”**



Seleccionamos con doble click la interfaz que deseamos configurar, en este caso la “eth0”



Y en el cuadro nos vamos a la pestaña **IPv6 Settings** y en **Method** seleccionamos “**Automatic, DHCP only**” y guardamos con el botón **Save**. Esto se puede hacer por consola pero la verdad es que este método es más rápido y lo mantenemos simple.



Y si ahora mandamos el comando **ip a** nuevamente podemos ver que nuestra interfaz (**eth0** en mi caso) ya tiene asignada una dirección IPv6.

```
(ozone@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether [redacted] brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.38/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0
        valid_lft 604280sec preferred_lft 604280sec
    inet6 [redacted]/64 scope global dynamic noprefixroute
        valid_lft 3599sec preferred_lft 3599sec
    inet6 [redacted]/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether [redacted] brd ff:ff:ff:ff:ff:ff
    inet 192.168.118.128/24 brd 192.168.118.255 scope global dynamic noprefixroute eth1
        valid_lft 952sec preferred_lft 952sec
    inet6 [redacted]/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

De todas formas este paréntesis sirvió para aprender a solucionar un pequeño inconveniente por medio del comando **dhclient** y ventanas graficas.

Ahora si podemos continuar a nuestro laboratorio...

Identificamos la interfaz que tiene nuestra IP, por ejemplo:

- **eth0** = 192.168.0.38 (*Bridge - la que habla con Ubuntu*)
- **eth1** = 192.168.118.128 (*Host only - la que habla con Metasploitable*).

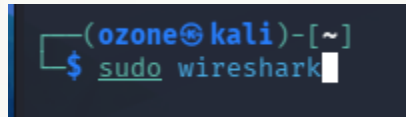
Usaremos:

- **eth0** para ver tráfico entre Ubuntu y Kali
- **eth1** para tráfico inseguro con Metasploitable

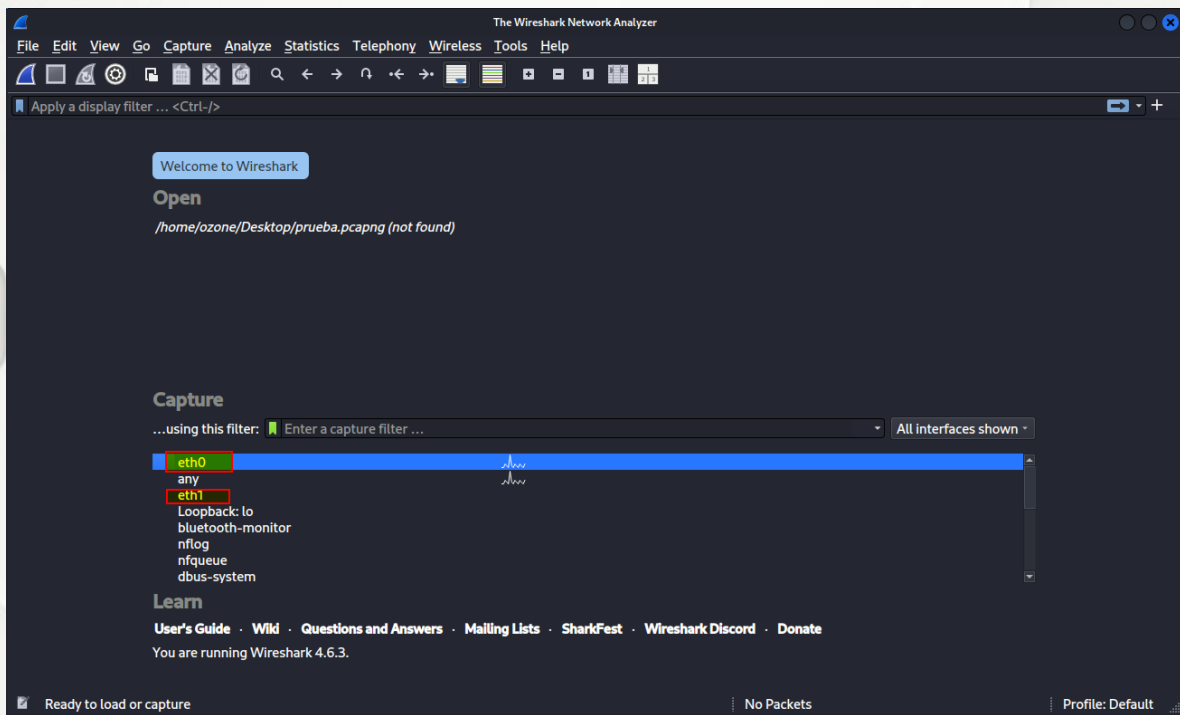
2. Abrir Wireshark en la interfaz correcta

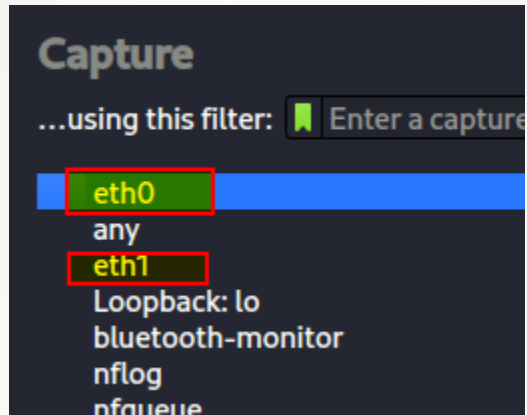
En Kali:

```
sudo wireshark
```



Y se nos abrirá Wireshark

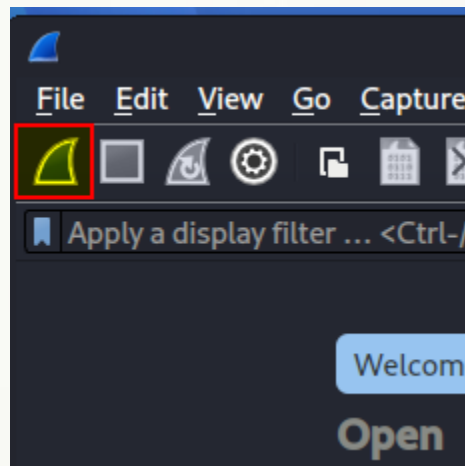




Seleccionamos:

- **eth0** para ver pings, SSH, etc. desde Ubuntu
- **eth1** para ver FTP y Telnet desde Metasploitable2

Para el siguiente punto seleccionamos la interfaz que se comunica con Ubuntu, en este caso la eth0 y le damos al botón capturar



3. ICMP (tráfico normal)

Como Kali no es router y estamos corriendo Wireshark en Kali, cualquier ping que lancemos desde la maquina Ubuntu hacia internet NO los veremos en nuestro sniffer de paquetes. Solo veremos paquetes dirigidos a Kali.

Para generarlos:

Desde Ubuntu:

```
ping IP_DE_KALI
```

Ejemplo:

```
ping 192.168.0.38
```

```
ozone@ubuntuserver:~$ ping 192.168.0.38
PING 192.168.0.38 (192.168.0.38) 56(84) bytes of data.
 64 bytes from 192.168.0.38: icmp_seq=1 ttl=64 time=1.30 ms
 64 bytes from 192.168.0.38: icmp_seq=2 ttl=64 time=1.23 ms
 64 bytes from 192.168.0.38: icmp_seq=3 ttl=64 time=1.32 ms
 64 bytes from 192.168.0.38: icmp_seq=4 ttl=64 time=1.46 ms

^C--- 192.168.0.38 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3006ms
 rtt min/avg/max/mdev = 1.232/1.327/1.463/0.084 ms
ozone@ubuntuserver:~$
```

En Wireshark:

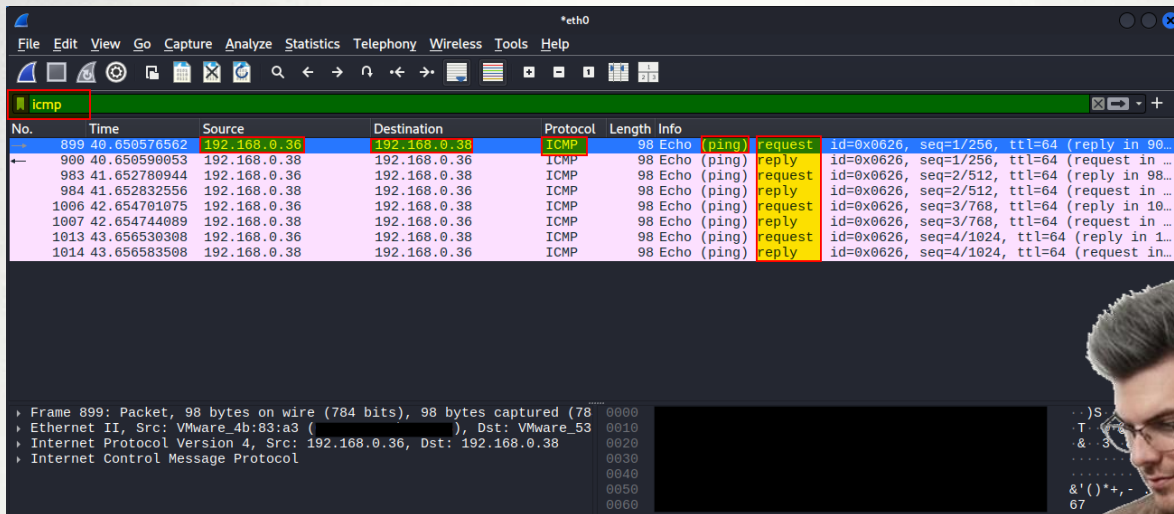
Detenemos la captura y filtramos:

```
icmp
```

Deberíamos ver:

- **Echo request** (Ubuntu > Kali)
- **Echo reply** (Kali > Ubuntu)

Listo, paquetes ICMP capturados correctamente.



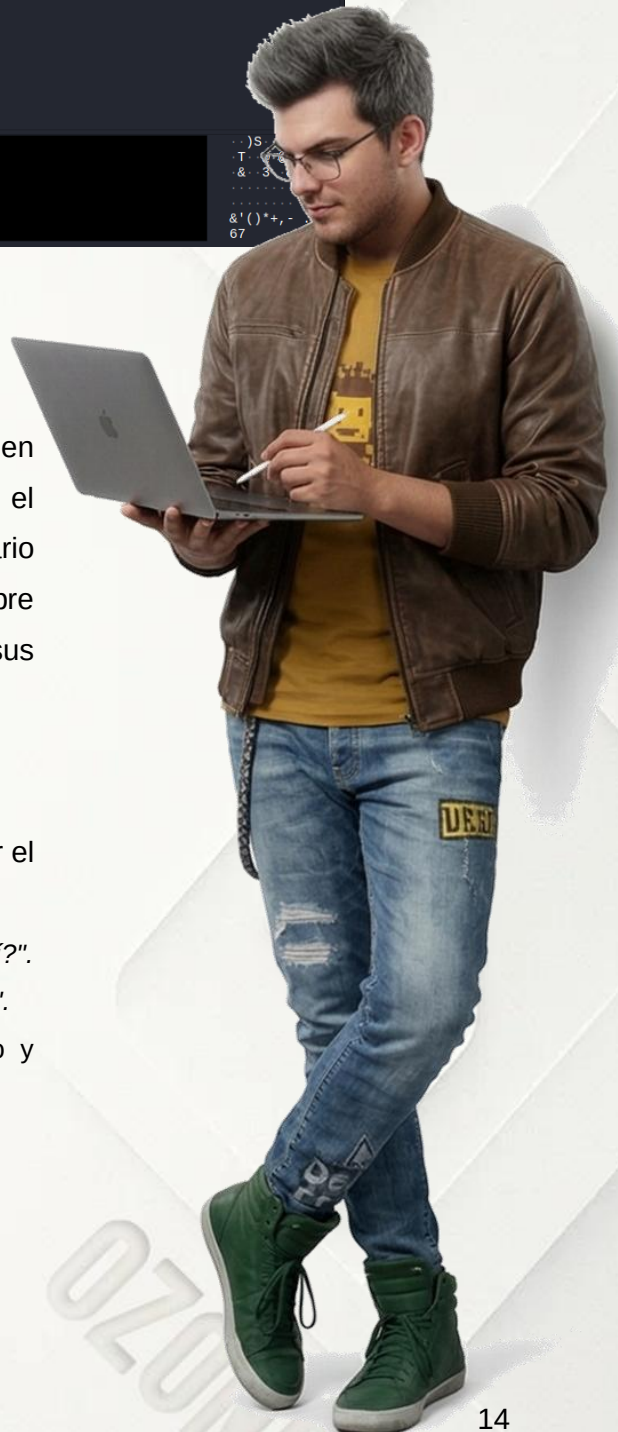
¿Y para que querría yo capturar paquetes ICMP?

Capturar paquetes **ICMP** (*Internet Control Message Protocol*) en Wireshark es una de las prácticas más fundamentales para el diagnóstico de redes. Aunque ICMP no transporta datos de usuario (como correos o páginas web), es el "mensajero" que informa sobre el estado y los errores de la comunicación. Aquí detallo sus utilidades principales:

Diagnóstico de conectividad (Ping)

Es el uso más común. Al capturar paquetes ICMP, podemos ver el ciclo completo de una solicitud:

- **Echo Request (Tipo 8):** Tu máquina preguntando "¿Estás ahí?".
- **Echo Reply (Tipo 0):** El destino respondiendo "Sí, aquí estoy".
- **Utilidad:** Permite medir el **RTT** (*Round Trip Time*) exacto y verificar si hay pérdida de paquetes en puntos específicos.



Identificación de errores de enrutamiento

ICMP avisa por qué no podemos llegar a un destino. En Wireshark, veremos paquetes con códigos específicos:

- **Destination unreachable (Tipo 3):** Indica que la red, el host, el puerto o el protocolo no están disponibles.
- **Time exceeded (Tipo 11):** Esto sucede cuando el **TTL** (*Time To Live*) llega a cero. Es lo que permite a herramientas como traceroute mapear cada salto de un paquete hasta su destino.

Detección de problemas de MTU (fragmentación)

Si un paquete es demasiado grande para un enlace y tiene activado el bit de "No Fragmentar" el router enviará un mensaje ICMP de tipo 3, código 4: **"Fragmentation Needed and DF set"**.

Utilidad: Capturar esto ayuda a ajustar el **MTU** de las interfaces para optimizar el rendimiento de la red.

Análisis de seguridad y escaneo

Los atacantes suelen usar ICMP para hacer un "mapeo" de la red. Capturarlos permite identificar:

- **ICMP sweeps:** Alguien intentando descubrir qué dispositivos están encendidos.
- **Redirecciones maliciosas (Tipo 5):** Un mensaje que le dice a nuestro host que envíe el tráfico a través de una ruta diferente (posiblemente controlada por un atacante).

Inspección del contenido del paquete

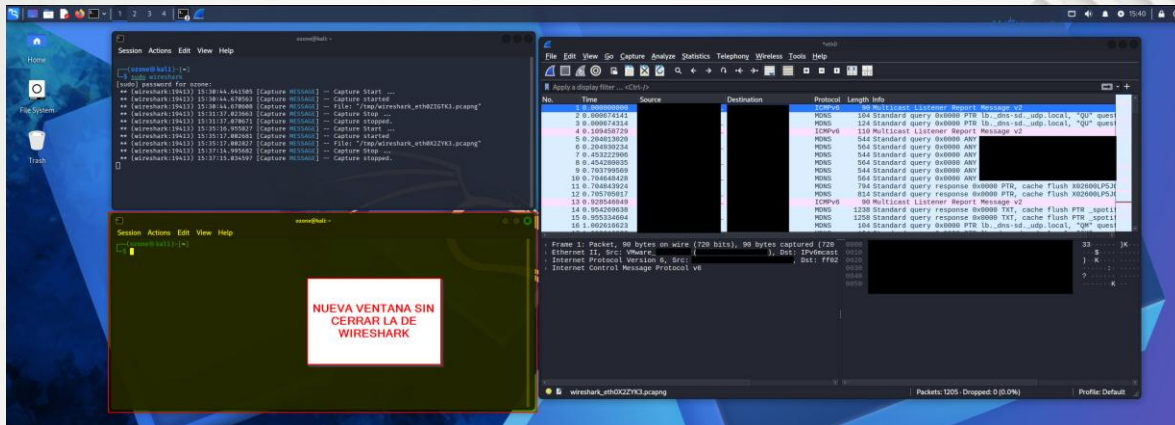
A diferencia de un simple comando en la terminal, Wireshark permite ver el **payload** del mensaje ICMP. A veces, los errores ICMP incluyen una copia de la cabecera del paquete IP original que causó el problema, lo que es invaluable para saber exactamente qué flujo de datos falló.

Resumen de tipos ICMP comunes en Wireshark

Tipo	Nombre	Función
0 / 8	Echo Reply / Request	Comprobar si un host está vivo (Ping).
3	Destination Unreachable	El paquete no pudo ser entregado.
5	Redirect	Sugiere una ruta más corta para el tráfico.
11	Time Exceeded	El paquete murió en el camino (TTL expirado).

4. SSH (tráfico cifrado)

Desde Kali hacia Ubuntu; abrimos una nueva terminal, sin cerrar la que tiene corriendo wireshark



iniciamos sesión por ssh en el servidor Ubuntu con:

```
ssh usuario@IP_UBUNTU
```

```
(ozone@kali)-[~]  
$ ssh ozone@192.168.0.36
```

```
ozone@ubuntuuserver:~$ whoami  
ozone  
ozone@ubuntuuserver:~$ ls  
lab  
ozone@ubuntuuserver:~$ pwd  
/home/ozone  
ozone@ubuntuuserver:~$
```

Podemos ver que Wireshark envía un **warning** pero no es nada que preocuparse,

```
[sudo] password for ozone:
** (wireshark:19413) 15:30:44.641505 [Capture MESSAGE] -- Capture Start ...
** (wireshark:19413) 15:30:44.670563 [Capture MESSAGE] -- Capture started
** (wireshark:19413) 15:30:44.670608 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0ZIGTK3.pcapng"
** (wireshark:19413) 15:31:37.023663 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:19413) 15:31:37.070671 [Capture MESSAGE] -- Capture stopped.
** (wireshark:19413) 15:35:16.955827 [Capture MESSAGE] -- Capture Start ...
** (wireshark:19413) 15:35:17.002681 [Capture MESSAGE] -- Capture started
** (wireshark:19413) 15:35:17.002827 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0X2ZYK3.pcapng"
** (wireshark:19413) 15:37:14.995682 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:19413) 15:37:15.034597 [Capture MESSAGE] -- Capture stopped.
** (wireshark:19413) 15:44:18.065303 [Capture MESSAGE] -- Capture Start ...
** (wireshark:19413) 15:44:18.107200 [Capture MESSAGE] -- Capture started
** (wireshark:19413) 15:44:18.107242 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0HEM2K3.pcapng"
** (wireshark:19413) 15:44:30.822666 [packet-ssh WARNING] ./epan/dissectors/packet-ssh.c:2393 -- ssh_set_kex_s
pacific_dissector(): NOT SUPPORTED OR UNKNOWN KEX DETECTED: ALGORITHM = sntrup761x25519-sha512@openssh.com
** (wireshark:19413) 15:45:18.473158 [Capture MESSAGE] -- Capture Stop ...
** (wireshark:19413) 15:45:18.510629 [Capture MESSAGE] -- Capture stopped.
```

¿Qué significa exactamente?

Wireshark nos dice que vio un **algoritmo de intercambio de claves (KEX)** que **no puede decodificar**:

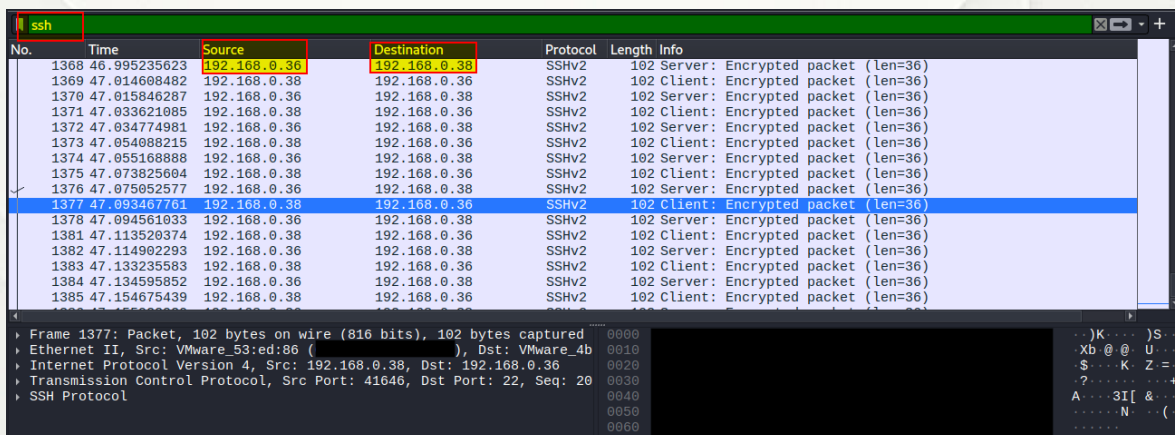
sntrup761x25519-sha512@openssh.com

Ese algoritmo es **nuevo**, moderno y diseñado por **OpenSSH** como parte de sus medidas **post-quantum**.

Wireshark **aun no soporta** la decodificación de ese KEX, por eso lanza el **warning**.

En Wireshark:

ssh



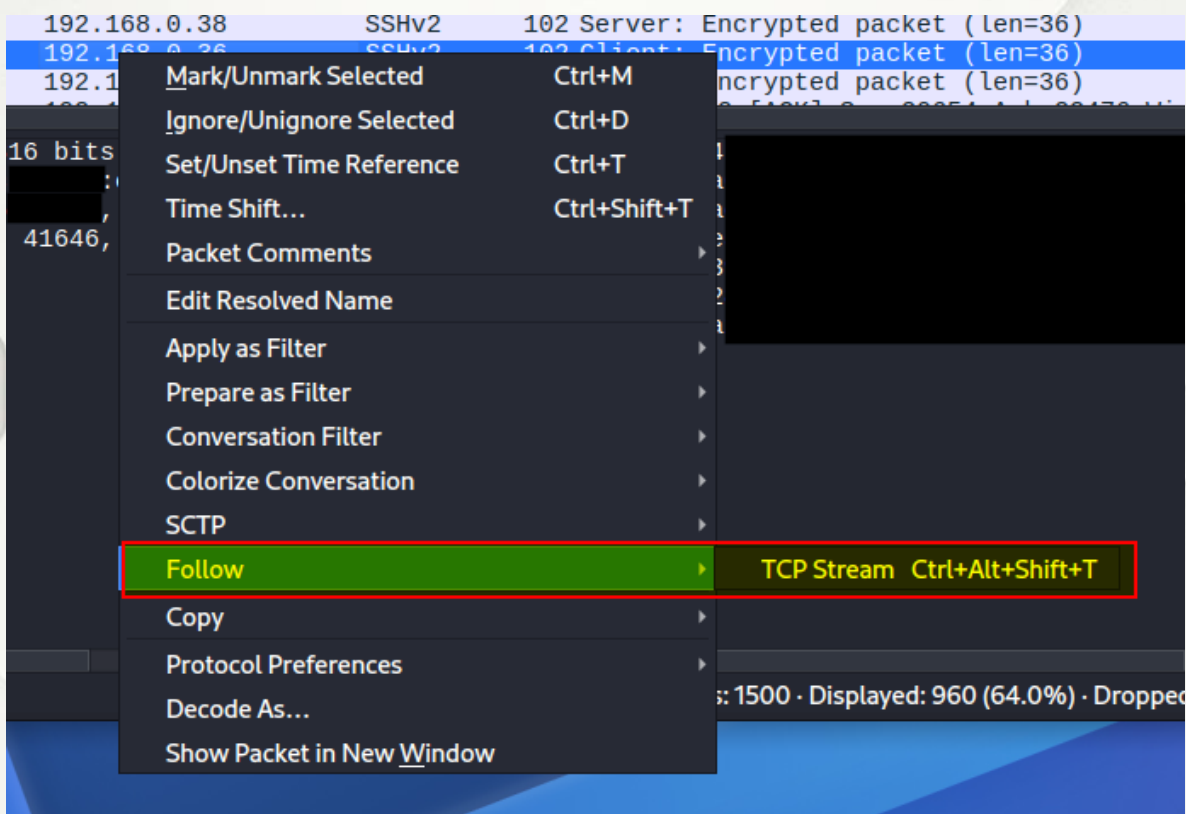
No.	Time	Source	Destination	Protocol	Length	Info
1368	46.995235623	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1369	47.014608482	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1370	47.015846287	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1371	47.033621085	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1372	47.034774981	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1373	47.054088215	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1374	47.055168888	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1375	47.073825604	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1376	47.075652577	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1377	47.093467761	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1378	47.094561033	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1381	47.113520374	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1382	47.114902293	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1383	47.133235583	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)
1384	47.134595852	192.168.0.36	192.168.0.38	SSHv2	102	Server: Encrypted packet (len=36)
1385	47.154675439	192.168.0.38	192.168.0.36	SSHv2	102	Client: Encrypted packet (len=36)

Frame 1377: Packet, 102 bytes on wire (816 bits), 102 bytes captured
Ethernet II, Src: VMware_53:ed:86 (), Dst: VMware_4b
Internet Protocol Version 4, Src: 192.168.0.38, Dst: 192.168.0.36
Transmission Control Protocol, Src Port: 41646, Dst Port: 22, Seq: 20
SSH Protocol

Vamos a ver:

- Handshake
- Paquetes "Encrypted"
- Ninguna credencial visible (esto es lo importante)

Si hacemos click derecho en cualquier paquete y vamos a **"Follow"**, **"TCP Stream"** podemos hacer el seguimiento completo de la conexión/conversación entre ambos equipos que envían ese paquete





Aunque en este caso nos sirve de poco debido a que ssh encripta todo el tráfico entre ambos equipos y no podremos ver nada en texto claro ni tampoco descifrar...¿Por qué?

SSH usa cifrado simétrico con llaves que nunca viajan por la red

Durante el handshake, cliente y servidor acuerdan una llave de sesión generada de forma segura.

Esa llave:

- No viaja por la red
- No puede ser derivada desde los paquetes
- Solo existe en RAM del cliente y del servidor

Entonces, aunque captures todos los paquetes:

No tienes la llave = no puedes descifrar.

La llave se genera con intercambio de claves (*Diffie-Hellman* o *X25519*)

Este método permite que:

- Ambos lados generen la misma llave
- Un atacante que escucha no pueda calcularla

Aunque tengamos el handshake completo:

Matemáticamente no se puede recuperar la llave.



5. FTP inseguro desde Metasploitable2

Captura en **eth1** (nuestra interfaz host-only).

En Kali:

```
ftp 192.168.56.102
```

Cuando nos pida usuario/clave:

- **Name:** msfadmin
- **Password:** msfadmin

```
(ozone@kali)~  
$ ftp 192.168.118.129  
Connected to 192.168.118.129.  
220 (vsFTPd 2.3.4)  
Name (192.168.118.129:ozone): msfadmin  
331 Please specify the password.  
Password:  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> ls
```

Ya estamos dentro del FTP inseguro.

Hagamos un par de comandos para generar tráfico:

- ls
- pwd
- cd
- tmp
- ls

```
ftp> ls  
229 Entering Extended Passive Mode (|||50555|).  
150 Here comes the directory listing.  
drwxr-xr-x  6 1000  1000      4096 Apr 28  2010 vulnerable  
226 Directory send OK.  
ftp> pwd  
Remote directory: /home/msfadmin  
ftp> cd tmp  
550 Failed to change directory.  
ftp> █
```

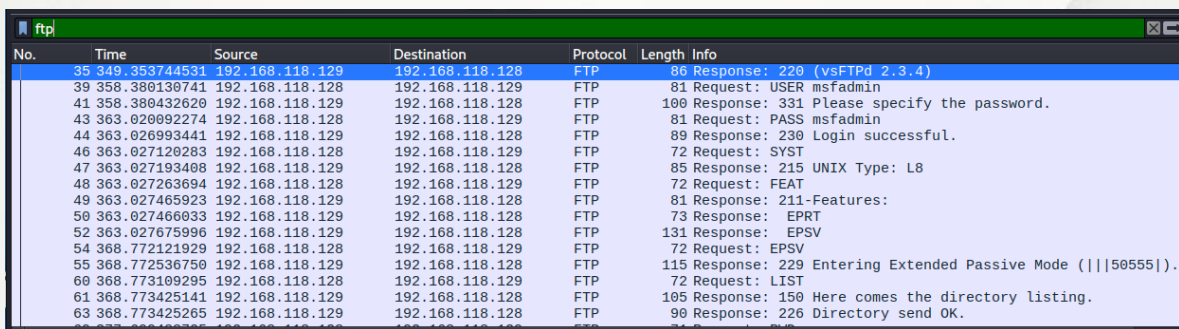
En Wireshark: aplicamos filtro

ftp

Deberíamos ver:

- USER msfadmin
- PASS msfadmin
- Comandos FTP (LIST, PWD, CWD, etc.)
- Todo en texto plano sin cifrado

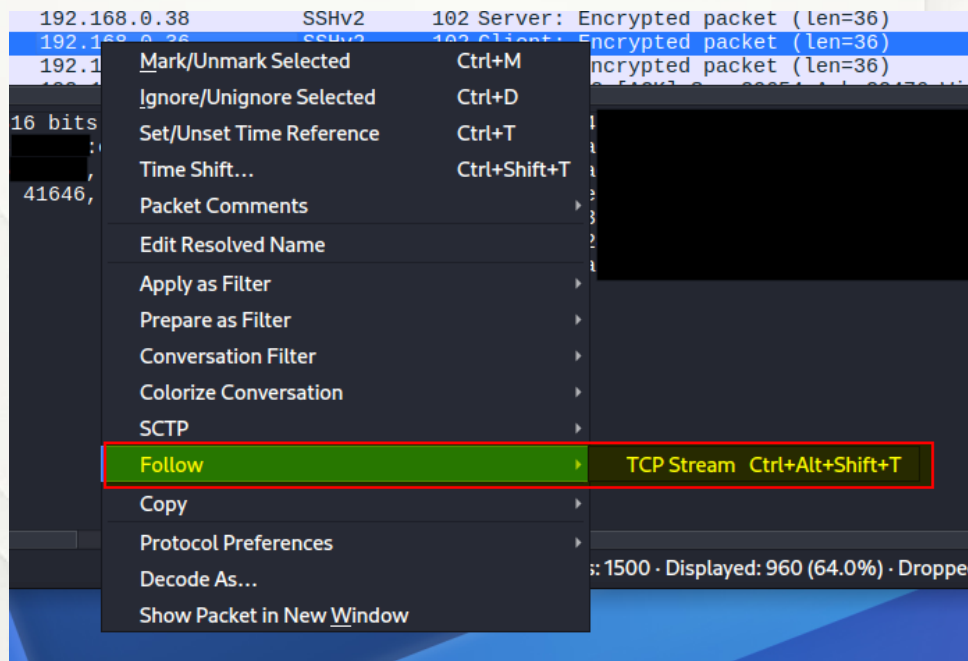
Este es el objetivo del laboratorio.



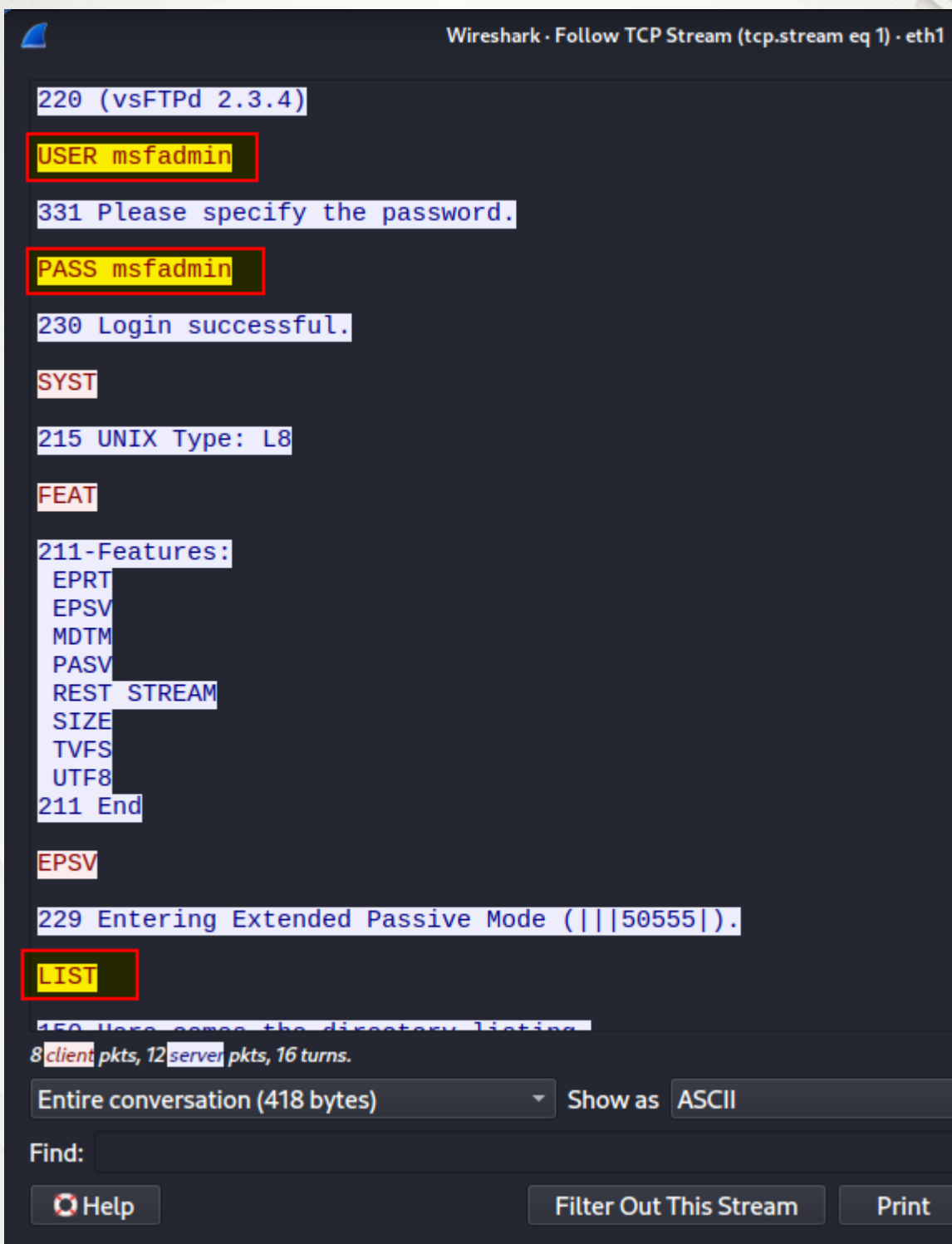
A screenshot of a Wireshark packet capture window titled 'ftp'. The packet list shows 63 packets. The first packet (No. 35) is an FTP Response (220) from 192.168.118.129 to 192.168.118.128. Subsequent packets show an FTP session: Request: USER msfadmin (No. 39), Response: Please specify the password (No. 41), Request: PASS msfadmin (No. 43), Response: Login successful (No. 44), Request: SYST (No. 46), Response: UNIX Type: L8 (No. 47), Request: FEAT (No. 48), Response: 211-Features (No. 49), Request: EPSV (No. 50), Response: EPRT (No. 52), Request: EPSV (No. 54), Response: Entering Extended Passive Mode (No. 55), Request: LIST (No. 60), Response: Here comes the directory listing (No. 61), and Directory send OK (No. 63).

No.	Time	Source	Destination	Protocol	Length	Info
35	349.353744531	192.168.118.129	192.168.118.128	FTP	86	Response: 220 (vsFTPd 2.3.4)
39	358.380138741	192.168.118.128	192.168.118.129	FTP	81	Request: USER msfadmin
41	358.380432620	192.168.118.129	192.168.118.128	FTP	100	Response: 331 Please specify the password.
43	363.020092274	192.168.118.128	192.168.118.129	FTP	81	Request: PASS msfadmin
44	363.026993441	192.168.118.129	192.168.118.128	FTP	89	Response: 230 Login successful.
46	363.027120283	192.168.118.128	192.168.118.129	FTP	72	Request: SYST
47	363.027193408	192.168.118.129	192.168.118.128	FTP	85	Response: 215 UNIX Type: L8
48	363.027263694	192.168.118.128	192.168.118.129	FTP	72	Request: FEAT
49	363.027465923	192.168.118.129	192.168.118.128	FTP	81	Response: 211-Features:
50	363.027466033	192.168.118.128	192.168.118.129	FTP	73	Request: EPRT
52	363.027675996	192.168.118.129	192.168.118.128	FTP	131	Response: EPSV
54	368.772121929	192.168.118.128	192.168.118.129	FTP	72	Request: EPSV
55	368.772536750	192.168.118.129	192.168.118.128	FTP	115	Response: 229 Entering Extended Passive Mode (50555).
60	368.773109295	192.168.118.128	192.168.118.129	FTP	72	Request: LIST
61	368.773425141	192.168.118.129	192.168.118.128	FTP	105	Response: 150 Here comes the directory listing.
63	368.773425265	192.168.118.128	192.168.118.129	FTP	90	Response: 226 Directory send OK.

Debemos recordar que si hacemos click derecho en cualquier paquete y vamos a **“Follow”**, **“TCP Stream”** podemos hacer el seguimiento completo de la conexión/conversación entre ambos equipos que envían ese paquete



En este caso podemos ver las credenciales y comandos en texto plano por que a diferencia de **ssh** , **ftp** no cifra la comunicación



Wireshark · Follow TCP Stream (tcp.stream eq 1) · eth1

```
220 (vsFTPd 2.3.4)
USER msfadmin
331 Please specify the password.
PASS msfadmin
230 Login successful.
SYST
215 UNIX Type: L8
FEAT
211-Features:
  EPRT
  EPSV
  MDTM
  PASV
  REST STREAM
  SIZE
  TVFS
  UTF8
211 End
EPSV
229 Entering Extended Passive Mode (|||50555|).
LIST
450 Here comes the directory listing.
8 client pkts, 12 server pkts, 16 turns.
```

Entire conversation (418 bytes) Show as ASCII

Find:

Help Filter Out This Stream Print

6. TELNET totalmente inseguro (también por eth1)

En Kali:

```
telnet IP_DE_METASPLOITABLE2
```

```
(ozone@kali)-[~]  
$ telnet 192.168.118.129  
Trying 192.168.118.129 ...  
Connected to 192.168.118.129.  
Escape character is '^['.
```

Obtener información del sistema

```
uname -a
```

Muestra kernel, arquitectura, etc.

Nombre del host

```
hostname
```

Listar archivos y directorios

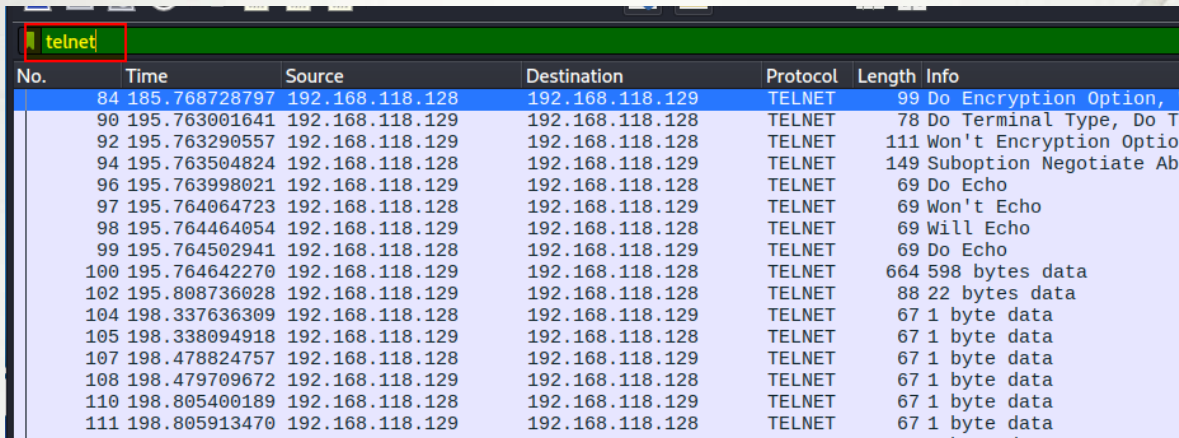
```
ls -la
```

Vamos a ver:

```
msfadmin@metasploitable:~$ uname -a  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux  
msfadmin@metasploitable:~$ hostname  
metasploitable  
msfadmin@metasploitable:~$ ls -la  
total 36  
drwxr-xr-x 5 msfadmin msfadmin 4096 2012-05-20 14:22 .  
drwxr-xr-x 6 root      root      4096 2010-04-16 02:16 ..  
lrwxrwxrwx 1 root      root        9 2012-05-14 00:26 .bash_history → /dev/null  
drwxr-xr-x 4 msfadmin msfadmin 4096 2010-04-17 14:11 .distcc  
-rw----- 1 root      root      4174 2012-05-14 02:01 .mysql_history  
-rw-r--r-- 1 msfadmin msfadmin  586 2010-03-16 19:12 .profile  
-rwx----- 1 msfadmin msfadmin    4 2012-05-20 14:22 .rhosts  
drwx----- 2 msfadmin msfadmin 4096 2010-05-17 21:43 .ssh  
-rw-r--r-- 1 msfadmin msfadmin    0 2010-05-07 14:38 .sudo_as_admin_successful  
drwxr-xr-x 6 msfadmin msfadmin 4096 2010-04-27 23:44 vulnerable  
msfadmin@metasploitable:~$
```

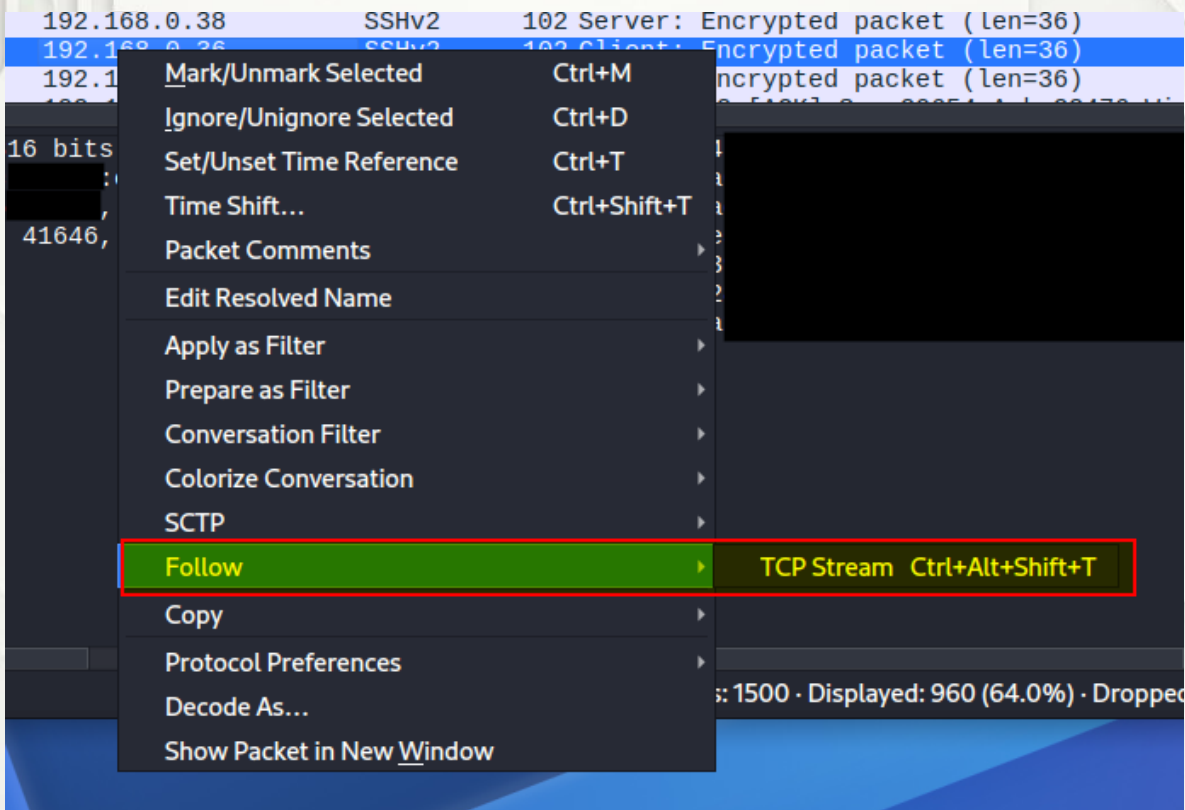
En Wireshark filtramos:

telnet

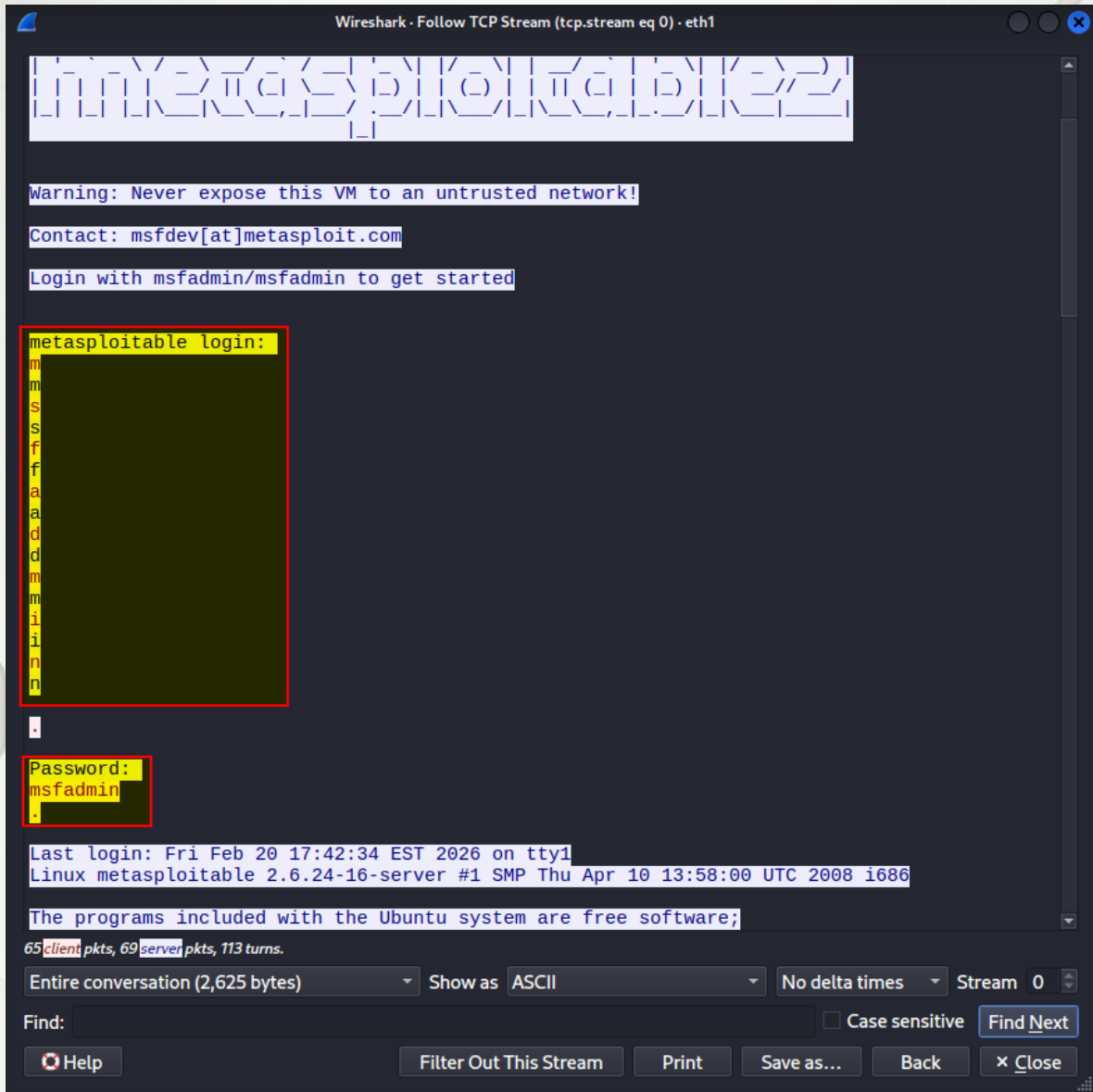


No.	Time	Source	Destination	Protocol	Length	Info
84	185.768728797	192.168.118.128	192.168.118.129	TELNET	99	Do Encryption Option, V
90	195.763001641	192.168.118.129	192.168.118.128	TELNET	78	Do Terminal Type, Do Te
92	195.763290557	192.168.118.129	192.168.118.128	TELNET	111	Won't Encryption Option
94	195.763504824	192.168.118.128	192.168.118.129	TELNET	149	Suboption Negotiate Abc
96	195.763998021	192.168.118.129	192.168.118.128	TELNET	69	Do Echo
97	195.764064723	192.168.118.128	192.168.118.129	TELNET	69	Won't Echo
98	195.764464054	192.168.118.129	192.168.118.128	TELNET	69	Will Echo
99	195.764502941	192.168.118.128	192.168.118.129	TELNET	69	Do Echo
100	195.764642270	192.168.118.129	192.168.118.128	TELNET	664	598 bytes data
102	195.808736028	192.168.118.129	192.168.118.128	TELNET	88	22 bytes data
104	198.337636309	192.168.118.128	192.168.118.129	TELNET	67	1 byte data
105	198.338094918	192.168.118.129	192.168.118.128	TELNET	67	1 byte data
107	198.478824757	192.168.118.128	192.168.118.129	TELNET	67	1 byte data
108	198.479709672	192.168.118.129	192.168.118.128	TELNET	67	1 byte data
110	198.805400189	192.168.118.128	192.168.118.129	TELNET	67	1 byte data
111	198.805913470	192.168.118.129	192.168.118.128	TELNET	67	1 byte data

Recordar que si hacemos click derecho en cualquier paquete y vamos a **“Follow”**, **“TCP Stream”** podemos hacer el seguimiento completo de la conexión/conversacion entre ambos equipos que envían ese paquete



Y podemos ver entre otras cosas en texto plano las credenciales de acceso:



The image shows a Wireshark packet capture window titled "Wireshark · Follow TCP Stream (tcp.stream eq 0) · eth1". The main display area shows a text-based login session. At the top, there is a large block of text that appears to be a banner or logo. Below this, there are several lines of text: "Warning: Never expose this VM to an untrusted network!", "Contact: msfdev[at]metasploit.com", and "Login with msfadmin/msfadmin to get started". The login prompt "metasploitable login:" is followed by a large block of text that appears to be a password being entered. Below this, the password "msfadmin" is displayed. The session then shows the last login time and the system version: "Last login: Fri Feb 20 17:42:34 EST 2026 on tty1" and "Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686". The bottom of the window shows the packet list and packet details pane, indicating that the stream is 2,625 bytes long and is shown in ASCII format.

```
Warning: Never expose this VM to an untrusted network!  
Contact: msfdev[at]metasploit.com  
Login with msfadmin/msfadmin to get started  
metasploitable login:  
msfadmin  
msfadmin  
Last login: Fri Feb 20 17:42:34 EST 2026 on tty1  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686  
The programs included with the Ubuntu system are free software;  
65 client pkts, 69 server pkts, 113 turns.  
Entire conversation (2,625 bytes) Show as ASCII No delta times Stream 0  
Find: Case sensitive Find Next  
Help Filter Out This Stream Print Save as... Back Close
```

- Usuario
- Contraseña
- Todo lo que se escribe
- Todo lo que responde el servidor

Esto demuestra un protocolo 100% inseguro. **¿Por qué?**

1. Son protocolos antiguos

FTP y **Telnet** fueron creados cuando internet era confiable y militar/académica. No existía el concepto de sniffing masivo.

2. No incluyen cifrado

No usan TLS, no usan SSL, no tienen ninguna capa de protección. Lo que escribimos se manda por la red así:

```
USER msfadmin
PASS msfadmin
ls
pwd
cat /etc/issue
```

Y cualquier equipo en la misma red los puede leer. Ni siquiera se molestan en enmascarar la contraseña.

3. No hay hashing, no hay TLS, no hay nada

Telnet literalmente envía:

```
m
s
f
a
d
m
i
n
```

carácter por carácter *FTP* envía:

```
USER msfadmin
PASS msfadmin
```

Para Wireshark es como leer un chat.

6.1 DIFERENCIA ENTRE PROTOCOLOS MODERNOS Y ANTIGUOS

Protocolo	Cifra todo	Contraseña cifrada	Seguro
FTP	No	No	No
Telnet	No	No	No
FTPS	Si (TLS)	Si	Si
SFTP	Si (SSH)	Si	Si
SSH	Si	Si	Si
HTTPS	Si	Si	Si

FTP y Telnet comparten la misma debilidad: **sin cifrado = texto plano**.



7. HTTP sin cifrado (opcional si Ubuntu tiene Apache)

Como no hemos instalado Apache hasta ahora en estos labs en nuestro servidor Ubuntu, lo instalaremos siguiendo estos sencillos pasos:

1. Instalar Apache en Ubuntu

En nuestra terminal de Ubuntu, ejecutamos estos comandos:

- Actualizar paquetes: `sudo apt update`
- Instalar Apache: `sudo apt install apache2 -y`

Para verificar que está corriendo, usamos: `systemctl status apache2`

2. Habilitar el tráfico (Configurar el Firewall)

Como hemos visto en laboratorios anteriores (Labs 3 y 4) Ubuntu usa un firewall llamado **UFW**. Debemos permitir el tráfico HTTP (puerto 80) para que otras máquinas puedan entrar.

- Permitir Apache: `sudo ufw allow 'Apache'`
- Activar el firewall: `sudo ufw enable` (si ya estaba activo, no es necesario).

¿Cómo saber mi IP?

Para poder consultarlo desde Kali, necesitamos la dirección IP de nuestro Ubuntu.

Escribir: `hostname -I` o `ip a` (En mi caso la IP es 192.168.0.36).

Y ahora ya podemos realizar solicitar con `curl` la página desde nuestro Kali a Ubuntu para generar tráfico.

En Kali:

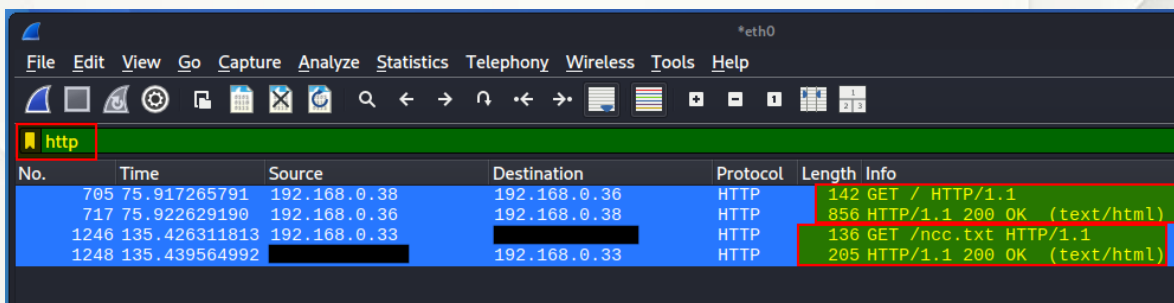
```
curl http://IP_UBUNTU
```

```
(ozone@kali)-[~]
$ curl http://192.168.0.36
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/
ional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
    * {
      margin: 0px 0px 0px 0px;
      padding: 0px 0px 0px 0px;
    }

    body, html {
      padding: 3px 3px 3px 3px;
```

En Wireshark:

http



The image shows a Wireshark capture on the eth0 interface. The packet list pane shows four packets. The first packet (No. 705) is a GET request from 192.168.0.38 to 192.168.0.36. The second packet (No. 717) is an HTTP 200 OK response from 192.168.0.36 to 192.168.0.38. The third packet (No. 1246) is a GET request from 192.168.0.33 to 192.168.0.33. The fourth packet (No. 1248) is an HTTP 200 OK response from 192.168.0.33 to 192.168.0.33.

No.	Time	Source	Destination	Protocol	Length	Info
705	75.917265791	192.168.0.38	192.168.0.36	HTTP	142	GET / HTTP/1.1
717	75.922629190	192.168.0.36	192.168.0.38	HTTP	856	HTTP/1.1 200 OK (text/html)
1246	135.426311813	192.168.0.33	192.168.0.33	HTTP	136	GET /ncc.txt HTTP/1.1
1248	135.439564992	192.168.0.33	192.168.0.33	HTTP	205	HTTP/1.1 200 OK (text/html)

Y como se puede apreciar vemos en los paquetes 705 el **GET** enviado desde Kali a Ubuntu solicitando la página con **curl** y en el 717 la respuesta del servidor Ubuntu con la página solicitada como se mostraba en la imagen anterior. Los otros 2 paquetes (1246 y 1248) corresponden a tráfico "extra" mientras hacíamos nuestras pruebas entre Kali y Ubuntu, muestran una conexión hacia un servidor externo que no tiene que ver con el laboratorio local.

Aquí explico qué está ocurriendo en esas tramas:

Análisis de las tramas 1246 y 1248

A diferencia de las primeras, donde el tráfico era entre la .38 (Kali) y la .36 (Ubuntu), este intercambio ocurre entre un dispositivo de mi red local y una IP pública:

- **Origen local:** 192.168.0.33 (un dispositivo con tarjeta de red **ASUSTeK**).
- **Destino remoto:** 190.46.255.43 (una IP pública, probablemente de un proveedor de internet o CDN).
- **Recurso solicitado:** El dispositivo intentó descargar un archivo llamado **ncc.txt** mediante una petición GET.

¿Qué es el archivo ncc.txt?

El archivo ncc.txt es comúnmente utilizado por sistemas operativos (especialmente por dispositivos de red o software específico) para una función llamada **Network Connectivity Center** o **Network Connectivity Check**.

Básicamente, el dispositivo hace lo siguiente:

1. Envía un GET /ncc.txt al servidor.
2. El servidor responde con un 200 OK.
3. Si el dispositivo recibe la respuesta correctamente, confirma que **tiene acceso a Internet** y que no está detrás de un "*Captive Portal*" (como los de los hoteles o aeropuertos que nos obligan a loguearnos).

En resumen

Lo que vemos es tráfico de fondo de mi propia red. Probablemente una computadora Asus (o una placa base Asus) en la IP .33 que, de forma automática, verificó si la conexión a internet estaba activa mientras hacíamos el **curl** en las otras máquinas.

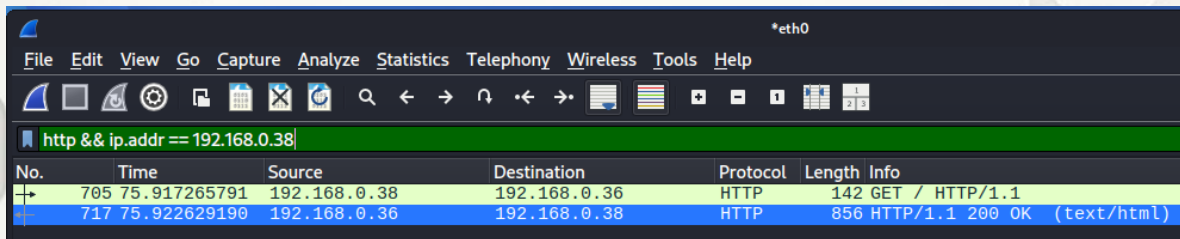
Si solo quisiéramos ver el tráfico asociado a nuestra máquina Kali en este caso, debemos eliminar del filtro aplicado, todo ese "**ruido**" extra que se genera y para esto utilizamos un filtro

compuesto que si entendemos la lógica es lo que abre la puerta a seguir entendiendo como filtrar paquetes en Wireshark. El filtro que aplicaremos es:

```
http && ip.addr == 192.168.0.36
```

http	filtra el protocolo que estamos buscando
&&	funciona como el “and”, básicamente está diciendo, quiero la condición “http” y también la siguiente condición
ip.addr ==	filtraremos por la dirección ip que es igual a...
192.168.0.36	la dirección ip que deseamos filtrar.

Y ahora solo nos aparecen los paquetes relacionados a los filtros aplicados.



Como dije anteriormente, esta es la lógica de los filtros aplicados en Wireshark, hay muchos recursos en internet donde dan ejemplos para búsquedas muy específicas y que están basadas en esta lógica. No tiene mucho sentido que explique cosas más específicas en relación a este tema en este documento ya que el objetivo es aprender y aplicar la herramienta más que dar una explicación exhaustiva de la misma que se aprende con la práctica.



Conclusión

Al finalizar este laboratorio, hemos transformado nuestra comprensión teórica de los protocolos en una evidencia visual y práctica. A través de la lente de Wireshark, hemos podido constatar tres realidades fundamentales de la ciberseguridad moderna:

1. **La invisibilidad del cifrado:** Confirmamos que protocolos como SSH protegen la integridad y confidencialidad de los datos mediante intercambios de claves complejos, haciendo que el esfuerzo de un atacante por interceptarlos sea matemáticamente inviable en condiciones normales.
2. **La vulnerabilidad del texto plano:** Visualizamos la peligrosidad de protocolos antiguos como FTP y Telnet. Ver una contraseña de administración aparecer directamente en nuestra pantalla nos recuerda por qué estos servicios deben ser erradicados o reemplazados por sus versiones seguras (SFTP/SSH) en entornos de producción.
3. **El dominio del entorno:** Aprendimos a filtrar el "*ruido*" de la red para centrar nuestra investigación en objetivos específicos, una habilidad crucial para cualquier analista de seguridad o administrador de sistemas durante un incidente.

Este ejercicio cierra el ciclo de aprendizaje sobre la infraestructura básica y nos prepara para desafíos más avanzados. Ahora que sabemos cómo se mueven los datos y qué tan expuestos pueden estar, tenemos una base sólida para realizar auditorías de red más profundas y proponer políticas de seguridad basadas en evidencias reales.